Project Planning Prompt for Antigravity

You are a senior AI Engineer, Cybersecurity Expert, Full-Stack Developer, and Software Architect. Your task is to design a complete, production-quality Final Year Engineering Project.

Project Title

AI-Powered Password Security Analyzer

Objective

Design and plan an intelligent password security platform that goes far beyond a traditional password strength checker. The system should use machine learning, cybersecurity principles, and an interactive web interface to analyze passwords, explain security risks, estimate resistance to attacks, and generate personalized recommendations.

The goal is to produce a project that is innovative, technically strong, visually impressive, and suitable for a final-year engineering demonstration.

Expected Deliverables

Provide a complete project plan including:

1. Problem Statement
2. Objectives
3. Novel Features
4. System Architecture
5. Technology Stack
6. Dataset Selection
7. Feature Engineering
8. Machine Learning Model Selection
9. Cybersecurity Logic
10. Database Design
11. API Design
12. Flask Application Structure
13. UI/UX Wireframe
14. Folder Structure
15. Development Roadmap
16. Testing Strategy
17. Deployment Plan
18. Future Enhancements

Do not write implementation code initially. Focus on planning and architecture.

Core Features

The system must include the following modules.

1. AI Password Strength Prediction

Use Machine Learning to classify passwords into:

- Weak
- Medium
- Strong
- Very Strong

Recommend the best ML algorithm after comparing:

- Random Forest
- XGBoost
- LightGBM
- CatBoost
- Logistic Regression
- Support Vector Machine

Explain why the selected model is the best.

---

2. Password Health Score

Generate a security score from **0–100** based on:

- Length
- Character diversity
- Entropy
- Dictionary words
- Repeated characters
- Keyboard patterns
- Sequential characters
- Password uniqueness
- Common password detection

Display the score using modern visual components such as progress bars and gauges.

---

3. Password Entropy Calculator

Calculate password entropy using standard information theory.

Display:

- Entropy value
- Security category
- Explanation of the result

---

4. Password Crack Time Estimation

Estimate resistance against different attack methods:

- Brute Force
- Dictionary Attack
- Hybrid Attack
- GPU Password Cracking
- Rainbow Table Attack (where applicable)

Display estimated crack times using user-friendly units such as seconds, minutes, days, years, or centuries.

---

5. Dictionary Word Detection

Detect:

- English dictionary words
- Common names
- Months
- Days
- Popular passwords
- Sports teams
- Cities
- Company names

Explain why using these words weakens a password.

---

6. Keyboard Pattern Detection

Detect patterns such as:

- qwerty

- asdfgh
- zxcvbn
- 123456
- abcdef
- qazwsx
- 1q2w3e

Also detect:

- Sequential characters
- Reverse sequences
- Repeated sequences
- Repeated characters

---

7. Common Password Detection

Compare passwords against leaked/common password datasets.

If a password is found:

- Warn the user immediately.
- Explain the associated risk.
- Recommend creating a new password.

The design must prioritize user privacy by avoiding storage of plaintext passwords.

---

8. Personalized Recommendations

Provide specific improvement suggestions.

Example:

- Increase length
- Add symbols
- Remove dictionary words
- Avoid predictable sequences
- Mix uppercase and lowercase letters
- Improve randomness

Do not give generic advice.

---

9. AI Strong Password Generator

Generate secure passwords based on user preferences:

- Desired length
- Memorable password
- Passphrase
- Banking
- Social media
- Email
- Gaming

Allow users to choose included character types.

Show the generated password's:

- Strength
- Entropy
- Estimated crack time

---

10. Interactive Dashboard

Design a modern Flask dashboard featuring:

- Password input
- Real-time analysis
- Animated strength meter
- Circular health score gauge
- Crack time visualization
- Entropy graph
- Character distribution chart
- Risk indicators
- Recommendation cards
- Password generator
- Analysis history

Use responsive design suitable for desktop and mobile.

---

11. Voice Input (Optional)

Allow password analysis through speech recognition for accessibility demonstrations.

---

12. Exportable PDF Report

Generate a professional report containing:

- Password strength
- Health score
- Entropy
- Crack time
- Risk analysis
- Improvement suggestions
- Charts
- Date and time
- System version

Bonus Features

Suggest additional innovative modules such as:

- Explainable AI (why the prediction was made)
- Password reuse detection
- Password similarity analysis
- Multi-language support
- Dark mode
- Accessibility features
- Offline analysis mode
- REST API
- Docker deployment
- Admin analytics dashboard
- User authentication
- Audit logs
- Password policy customization
- Rate limiting
- Secure session management

Technical Requirements

Recommend an optimal technology stack.

Possible stack:

Backend:

- Python
- Flask

Machine Learning:

- Scikit-learn
- XGBoost
- LightGBM

Frontend:

- HTML5
- CSS3
- Bootstrap 5
- JavaScript
- Chart.js

Database:

- SQLite (development)
- PostgreSQL (production-ready option)

Visualization:

- Plotly
- Chart.js

PDF Generation:

- ReportLab

Deployment:

- Docker
- Nginx
- Gunicorn
- GitHub

Project Architecture

Design a scalable architecture using clear modules, including:

- Authentication
- Password Analysis Engine

- Feature Engineering Engine
- ML Prediction Engine
- Risk Assessment Engine
- Recommendation Engine
- Password Generator
- Reporting Module
- Dashboard
- REST API
- Database Layer

Provide a modular folder structure that follows software engineering best practices.

Security Requirements

The application must:

- Never store plaintext passwords.
- Perform all analysis securely.
- Use hashing only when persistence is required.
- Protect against XSS, CSRF, and SQL Injection.
- Follow secure coding practices.
- Respect user privacy.

Machine Learning Requirements

Explain:

- Dataset selection
- Data preprocessing
- Feature engineering
- Feature importance
- Model comparison
- Hyperparameter tuning
- Cross-validation
- Evaluation metrics (Accuracy, Precision, Recall, F1 Score, ROC-AUC where applicable)
- Model persistence
- Prediction workflow

User Experience

The application should look like a modern cybersecurity product rather than a student project.

Aim for:

- Professional branding
- Smooth animations
- Responsive layout
- Intuitive navigation
- Attractive visualizations
- Clear explanations for non-technical users

Final Deliverable

Produce a detailed implementation roadmap that breaks the project into manageable phases with milestones, estimated effort, dependencies, and risks. Include recommendations for optional advanced features if time permits.

The resulting project should demonstrate strong software engineering, machine learning, cybersecurity, UI/UX design, and deployment practices, making it suitable for a final-year engineering showcase and an impressive portfolio project.